

ÉVALUATION FINALE

Développement Web avec Symfony

Projet : BlogTech

Plateforme de blog collaboratif avec API REST

Consignes générales

❑ Important — À lire avant de commencer

- Lisez l'intégralité du sujet avant de commencer à coder.
- Le projet doit être réalisé individuellement.
- Committez régulièrement votre travail avec des messages de commit clairs.
- La qualité du code (lisibilité, respect des conventions Symfony) est évaluée.
- Les parties sont indépendantes : si vous bloquez, passez à la suivante.
- Tout plagiat ou utilisation d'un outil d'IA génératif entraîne la note de 0.

Prérequis techniques

- PHP 8.3+ et Composer installés
- Symfony CLI (recommandé) ou serveur Apache/Nginx
- MySQL ou MariaDB pour la base de données

Structure du rendu attendu

Rendez moi votre projet comme d'habitude avec un lien vers votre dépôt GIT + Ce document Word rempli avec les questions de cours 😊

Présentation du projet — BlogTech

Vous allez développer BlogTech, une plateforme de blog collaboratif permettant à des utilisateurs de publier des articles, de les organiser par catégories et de les commenter. La plateforme expose également une API REST pour permettre à des applications tierces de consulter les articles publiés (on ne développera pas ces applications tierces, seulement l'API)

Fonctionnalités attendues

- Connexion des utilisateurs (authentification sécurisée)
- Espace d'administration (back-office) pour gérer les articles et les catégories et modérer les commentaires
- Interface publique (front-office avec TWIG) pour consulter les articles publiés
- Système de commentaires sur les articles
- API REST pour exposer les articles publiés

Modèle de données

Voici le schéma des entités à implémenter, y compris l'entité User qui sera à créer en mettant en place l'authentification :

Entité	Champs principaux	Relations (et les champs qui les représentent)
User	id, email, password, roles, username, createdAt	OneToMany → Article (propriété articles) OneToMany → Comment (propriété comments)
Article	id, title, content, slug, status (draft/published), createdAt, updatedAt	ManyToOne → User (propriété author) ManyToMany → Category (propriété categories) OneToMany → Comment (propriété comments)
Category	id, name, slug	ManyToMany → Article (propriété articles)
Comment	id, content, createdAt, approved	ManyToOne → Article (propriété article) ManyToOne → User (propriété author)

Partie 1 — Configuration & Modèle de données

1.1 — Initialisation du projet

1. Créez un nouveau projet Symfony en utilisant la commande appropriée
2. Paramétrez votre fichier `compose.yaml` pour pouvoir démarrer un conteneur de base de données grâce à Docker. Nommez bien votre volume `[prenom]_blogtech_db_data`
3. Configurez la connexion à la base de données dans le fichier `.env.local`
4. Installez les bundles nécessaires au développement du projet grâce à composer

1.2 — Création des entités

Créez les quatre entités décrites dans le modèle de données ci-dessus avec les **Assertions** suivantes :

- User : email valide, mot de passe d'au moins 8 caractères, username non vide (**utilisez bien la méthode vu dans le cours sur l'authentification pour créer cette entité, ça sera plus simple**)
- Article : title entre 5 et 255 caractères, content non vide, slug unique. Pour la propriété status, utilisez le type de propriété « **enum** » lorsque le MakerBundle va vous demander quel est le type de la propriété souhaité, et donnez lui comme chemin `App\Enum\StatusEnum`. Il faudra au préalable créer le fichier `src/Enum/StatusEnum.php` qui contiendra ceci :

```
<?php
// src/Enum/StatusEnum.php
namespace App\Enum;

enum StatusEnum: string
{
    case Draft = 'draft';
    case Published = 'published';
}
```

- Si vous n'arrivez pas à mettre en place le type « Enum », restez sur un type string, ne vous bloquez pas sur ça ☺ *
- Category : name unique, slug unique
- Comment : content non vide, approved = booléen false par défaut (pas NULL, FALSE !)

LISTE DES ASSERTIONS POSSIBLES : <https://symfony.com/doc/current/validation.html>

Générez et exécutez les migrations pour créer la base de données.

Puis intégrez les fixtures que je vous ai fourni. Attention lorsque vous nommez vos propriétés, nommez-les bien comme dans l'énoncé sinon ma fixture risque de ne pas fonctionner.

*Si vous n'avez pas intégré le type « Enum » sur la propriété « status » de l'entité « Article », il faudra alors modifier la Fixture pour pouvoir l'intégrer. Ici, vous avez `StatusEnum::Published` qu'il faudra remplacer par « published » si vous l'avez laissé en type string par exemple.

```
32 // --- Articles ---
33 $articlesData = [
34     [
35         'title'      => 'L\'intelligence artificielle en 2025',
36         'content'    => 'L\'IA transforme tous les secteurs d\'activité à une vitesse sans précédent.',
37         'slug'       => 'intelligence-artificielle-2025',
38         'status'     => StatusEnum::Published,
39         'authorRef'  => 'user_admin',
40         'categories' => [0],
41     ],
42 ];
```

Partie 2 — Authentification & Sécurité

2.1 — Système d'authentification

1. Configurez le SecurityBundle pour utiliser l'entité User à l'authentification.
2. Implémentez le formulaire de connexion (LoginController) en utilisant le système d'authentification natif de Symfony (form_login).
3. Implémentez la déconnexion également sur la route /logout

2.2 — Contrôle d'accès

Configurez les règles d'accès suivantes dans security.yaml :

- Les routes préfixées par /admin ne sont accessibles qu'aux utilisateurs ayant le rôle `ROLE_ADMIN`.
- Le reste du site est accessible publiquement.

Partie 3 — Back-office d'administration

TOUTE CETTE PARTIE PEUT (DOIT) ÊTRE TRAITÉE AVEC LE MAKERBUNDLE
(make:crud)

3.1 — Gestion des catégories

Créez un CRUD complet pour les catégories (accessible uniquement aux administrateurs) :

- Liste des catégories en affichant dans une des colonnes de la vue liste (index) le nombre d'articles associés
- Formulaire de création / modification (CategoryType) avec génération automatique du slug à partir du nom
- Suppression d'une catégorie

3.2 — Gestion des articles

Créez un CRUD complet pour les articles (accessible aux administrateurs) :

- Liste des articles avec titre, auteur, statut et date de création formatée d/m/Y
- Formulaire de création / modification (ArticleType) incluant : titre, contenu (textarea), sélection des catégories (choix multiple en reprenant le nom de la catégorie dans le select), statut
- L'auteur d'un article est automatiquement défini comme l'utilisateur connecté lors de la création. Pour cela vous pouvez utiliser `$this->getUser()` dans le contrôleur
- Suppression d'un article

3.3 — Modération des commentaires

Créez une interface de modération des commentaires permettant à l'administrateur d'approuver ou de supprimer les commentaires en attente. Pour cela vous pouvez simplement utiliser `make:crud` pour obtenir la vue liste des commentaires, puis utiliser un `<button>` pour changer le champ « **approved** » dans la vue « édition ». Pour cela vous pouvez utiliser **stimulus** pour détecter l'évènement de clic sur le `<button>` et envoyer une requête vers une route prévue à cet effet : C'est le même fonctionnement que de l'AJAX, mais sans jQuery ☺

Indice : Prévoyez une Route PATCH dans CommentController.php qui ressemblerait à ceci :

```
#[Route('/admin/comments/{id}/toggle-status', name: 'toggle_status', methods: ['PATCH'])]
```

Et enfin, vous pouvez lancer un appel XMLHttpRequest asynchrone depuis votre controller javascript en reprenant ce code :

```
const response = await fetch(ICI_VOTRE_ROUTE_CONTENANT L'ID (URL), {
  method: 'PATCH',
  headers: {
    'X-Requested-With': 'XMLHttpRequest',
    'Accept': 'application/json',
  },
});
```

```
// if (!response.ok) throw new Error('Erreur réseau');  
if (!response.ok) {  
    throw new Error(`Erreur ${response.status}`);  
}
```

```
UTILISEZ UN CONSOLE.LOG DE DATA POUR SAVOIR QUOI FAIRE DU RESULTAT  
const data = await response.json();
```

Partie 4 — Front-office public & Twig

4.1 — Layout et templates

1. Créez un template de base `base.html.twig` incluant : une barre de navigation (liens vers l'accueil, les catégories (Vous pouvez utiliser une extension Twig ici pour générer vos liens de menu 😊), la connexion/déconnexion selon l'état de l'utilisateur), un bloc **content** et un pied de page.
2. Créez un **partial** Twig réutilisable pour afficher les données des articles (titre, extrait du contenu (100 premiers caractères), auteur, date, catégories), et vous allez utiliser ce **partial** pour chaque article dans la liste des articles : je veux une boucle 😊

4.2 — Routes publiques

- Page d'accueil (**URL : /**) : liste des articles publiés, du plus récent au plus ancien.(basez vous sur la date de création, pas de mise à jour)
- Page de détail d'un article (**URL : /article/{id}**) : affichage complet de l'article, de ses catégories et de ses commentaires approuvés.
- Page de liste par catégorie (**URL : /category/{slug}**) : affiche tous les articles publiés d'une catégorie donnée.

Partie 5 — API REST

Créez une API REST permettant à des applications tierces de consulter les articles publiés.

5.1 — Endpoints

Méthode	URL	Description
GET	/api/articles	Retourne la liste des articles publiés (id, title, slug, author.username, createdAt, categories)
GET	/api/articles/{id}	Retourne le détail complet d'un article publié (y compris les commentaires)
GET	/api/categories	Retourne la liste des catégories avec leurs articles

5.2 — Contraintes techniques

- Utilisez le composant Serializer de Symfony avec des groupes de sérialisation (attribut #[Groups]) pour éviter les références circulaires.
- L'API est publique (aucune authentification requise pour la lecture, donc pas de JWT)

❏ Conseils de gestion du temps

- Commencez par les parties 1 et 2, elles sont fondamentales pour la suite.
- Si vous manquez de temps, priorisez une fonctionnalité complète et fonctionnelle plutôt que plusieurs incomplètes.
- Pensez à tester votre application régulièrement (navigateur + Swagger pour tester l'API).
- Un README clair et des commits bien nommés sont valorisés dans la notation.

Partie 6 — Cours

Dans cette partie, vous pouvez oublier le projet BlogTech, je vais simplement vous poser des questions de cours pour lesquelles j'attends des explications et des définitions. Vous pouvez répondre directement dans ce document, ou dans un fichier .txt à part si vous le souhaitez. Dans tous les cas il faudra bien numéroter vos réponses pour que je puisse savoir à quelle question vous répondez, et surtout nommez bien votre fichier

[prenom]-ecf3.docx ou [prenom]-ecf3.txt

C'est parti 😊

Question 1 :

Qu'est-ce qui différencie un test unitaire d'un test fonctionnel ? Donnez un exemple de test unitaire.

Un test unitaire consiste en une vérification sur une partie isolée du code, sortie de son contexte. On va par exemple vérifier qu'une méthode renvoie toujours quelque chose, qu'elle se comporte bien si je lui envoie de mauvais paramètres, et qu'elle ne casse rien le cas échéant.

Un test fonctionnel, lui, va consister en la vérification du fonctionnement complet d'une feature, de l'appel de la route http jusqu'à la réponse.

Question 2 :

L'API que nous avons mis en place sur BlogTech dans la partie Programmation de cette évaluation ne possède que des endpoints destinés à la lecture. Les groupes de serialisation me permettent de **restreindre** ce qui est envoyé en **lecture**. Que dois-je mettre en place si je veux restreindre ce que l'endpoint peut recevoir en **écriture** ? Nommez l'outil, donnez sa définition et donnez un exemple.

Pour être certains de ne pas laisser passer des données imprévues ou mal formatées jusqu'à l'API, on peut utiliser un DTO qui se place entre le client et l'API afin de contrôler la structure des payloads envoyés à l'API. Si la structure ou les données reçues ne correspondent pas au cas d'utilisation prévu, alors le DTO rejette purement et simplement l'appel : l'API n'est pas sollicitée.

Il est également possible d'utiliser le `denormalizationContext` afin de surveiller ce qu'il est possible d'écrire grâce à l'API, mais cette méthode présente le désavantage de déléguer à l'API la vérification de chaque champ individuellement et les messages d'erreur correspondant alors qu'on ne devrait pas la solliciter avec un payload non conforme.

La solution idéale consiste certainement en l'utilisation des deux.

Question 3 :

Comment appelle-t-on ce format de données enrichi ? :

```
"categories": [  
  {  
    "@id": "/api/categories/1",  
    "@type": "Category",  
    "name": "Technologie"  
  }  
],
```

Que puis-je faire et dans quel fichier si je veux récupérer mes données au format JSON « classique » ?

Ce format enrichi s'appelle le **JSON+LD** et consiste en la présence d'IRI rajoutés par-dessus les données réelles du JSON de base, permettant à l'API d'obtenir des informations et des correspondances de ressources de manière optimale.

Dans `/config/packages/api_platform.yaml`, on peut préciser quel est le format par défaut souhaité en le plaçant en premier dans la clé « formats ». Si l'on souhaite proposer plusieurs formats de donnée et laisser le choix à l'utilisateur, il pourra alors préciser dans les headers de sa requête le format souhaité.

Question 4 :

Que fait exactement ce bout de code et à quoi sert-il ? Utilisez l'exemple de la capture pour expliquer, nous sommes dans l'entité Ordonnance :

```
#[ApiResource(  
    operations: [  
        new GetCollection(  
            normalizationContext: ['groups' => ['ordonnance:readlist']]  
        ),  
        new Get(  
            normalizationContext: ['groups' => ['ordonnance:readdetail']]  
        )  
    ],  
)]
```

Ce code nous permet d'exposer l'entité Ordonnance sur deux opérations distinctes : `GetCollection` et `Get`, en leur assignant à chacune un groupe de serialization.

Ces groupes nous permettront par la suite de déterminer ce qui doit s'afficher de l'entité et dans quel contexte.

Question 5 :

Que fait cette fonction ? Nous sommes dans un fichier qui s'appelle CabinetType.php
Où se trouve ce fichier ? Est-il utilisé dans le projet Doctofree ?

```
public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('nom')
        ->add('adresse')
        ->add('telephone')
        ->add('medecins', EntityType::class, [
            'class' => Medecin::class,
            'choice_label' => 'nom',
            'multiple' => true,
        ])
    ;
}
```

Ce fichier sert à détailler quels seront les champs d'un formulaire pour cette entité donnée. Il se trouve dans le répertoire src/Form et NON, il n'est pas utilisé dans Doctofree puisque nous n'avons plus de formulaires dans Doctofree. Il a simplement été généré lors de la commande make:crud et n'a pas été supprimé.

Question 6 :

Que se passe-t-il dans ce fichier exactement ? (templates/admin/article/_form.html.twig)

```
templates > admin > article > _form.html.twig
1  {{ form_start(form) }}
2  {{ form_widget(form) }}
3  <button class="btn">{{ button_label|default('Save') }}</button>
4  {{ form_end(form) }}
5
```

Ce fichier permet de générer le formulaire correspondant à l'entité Article en se basant sur le contenu du fichier src/Form/ArticleType.php

La ligne 2 va parcourir le FormBuilder du fichier ArticleType.php et générer les champs adéquats dans le HTML du template.

Question 7 :

Analysez le test Unitaire suivant et expliquez ce qu'il fait. Donnez-moi le nom que l'on donne au **pattern** de test qui est visible ici.

```
$rdv = new Rendezvous();

$this->patient->addListeRendezVou($rdv);
$this->patient->addListeRendezVou($rdv);

$this->assertCount(1, $this->patient->getListeRendezVou());
```

Ce test permet de vérifier qu'un patient ne peut pas se voir attribuer deux fois le même Rendez-Vous. Pour cela on ajoute exprès le même rendez-vous deux fois, et on vérifie la taille du tableau après. Si le tableau est égal à 1, alors c'est que le doublon a été rejeté et donc le test est concluant, sinon le test est échoué.

Ce pattern s'appelle AAA, pour Arrange, Act, Assert. Dans l'ordre on prépare les données (Arrange), on effectue une action donnée (Act), puis on vérifie le résultat (Assert)

Question 8 :

Dans un controller stimulus, j'ai déclaré la valeur « **total** », et j'y ai écrit cette fonction :

```
totalValueChanged() {
    this.mettreAJourCompteur();
    this.mettreAJourProgression();
}
```

Que fait cette fonction ? Quand est-elle appelée ? Comment appelle-t-on ce genre de fonctions ?

Cette fonction se déclenche lorsque la valeur « **total** » est modifiée. On appelle cela un **callback**, et cette fonction en particulier lance deux appels de fonction pour mettre à jour l'UI lorsqu'un changement est détecté sur la valeur « **total** ».

Question 9 :

Oups, ma migration semble ne pas fonctionner, pourtant la génération s'est bien passé. Que se passe-t-il ?

```
nicolas@MUSS:~/wsldev/projects/premier_projet$ php bin/console doctrine:migrations:migrate
In ExceptionConverter.php line 91:

An exception occurred in the driver: SQLSTATE[HY000] [1045] Access denied for user 'symfodny'@'172.24.0.1' (using password: YES)

In Exception.php line 24:

SQLSTATE[HY000] [1045] Access denied for user 'symfodny'@'172.24.0.1' (using password: YES)

In PDOConnect.php line 25:

SQLSTATE[HY000] [1045] Access denied for user 'symfodny'@'172.24.0.1' (using password: YES)

doctrine:migrations:migrate [--write-sql [WRITE-SQL]] [--dry-run] [--query-time] [--allow-no-migration] [--all-or-nothing [ALL-OR-NOTHING]] [--no-all-or-nothing]
[--configuration CONFIGURATION] [--em EM] [--conn CONN] [--] [<version>]
```

Cette erreur me permet d'être sûr qu'il s'agit d'une erreur d'authentification, et en analysant le message en console, on voit que l'utilisateur s'appelle symfodny. Probablement une faute de frappe dans le fichier .env.local donc.

Question 10 :

Mon template ici présent ne produit pas les résultat attendu dans mon projet Symfony : j'ai beau regarder le code je ne vois pas d'erreur, et Symfony me dit que « monService » n'existe pas... Que se passe-t-il ?

```
<div id="services">
  <div class="container">
    <h2 class="text-white text-center text-uppercase">Nos services</h2>
    <div class="row">
      @for(monService of services; track monService.titre) {
        <div class="col-12 col-md-6 col-lg-4 my-4 text-center item">
          <img [src]="monService.picto" class="img-fluid"/>
          <h3 class="text-white text-uppercase">
            {{ monService.titre }}
          </h3>
          <h4 class="text-white text-uppercase">
            {{ monService.soustitre }}
          </h4>
          <p class="text-white">
            {{ monService.texte }}
          </p>
        </div>
      }
    </div>
  </div>
</div>
```

C'est du code Angular 😊

Annexes

A — Commandes Symfony utiles

Commande	Description
<code>php bin/console make:entity</code>	Créer ou modifier une entité
<code>php bin/console make:migration</code>	Générer une migration
<code>php bin/console doctrine:migrations:migrate</code>	Exécuter les migrations
<code>php bin/console make:fixtures</code>	Créer une fixture
<code>php bin/console doctrine:fixtures:load</code>	Charger les fixtures
<code>php bin/console make:controller</code>	Créer un contrôleur
<code>php bin/console make:form</code>	Créer un type de formulaire
<code>php bin/console make:security:form-login</code>	Configurer l'authentification par formulaire
<code>php bin/console debug:router</code>	Afficher toutes les routes
<code>php bin/console cache:clear</code>	Vider le cache

B — Structure de réponse JSON attendue pour l'API

Exemple de réponse pour GET `/api/articles/{slug}` :

```
{
  "id": 1,
  "title": "Introduction à Symfony",
  "slug": "introduction-a-symfony",
  "content": "...",
  "createdAt": "2025-05-14T10:00:00+00:00",
  "author": { "username": "johndoe" },
  "categories": [{ "name": "PHP", "slug": "php" }],
  "comments": [
    { "content": "Super article !", "createdAt": "...", "author": { "username": "jane" } }
  ]
}
```